

STAT 240

Week 6: Regression and Intro to APIs

Owen G. Ward

This week

- More details on linear regression
- Getting data from an API
 - The eBird API
- Quick midterm review

Last time - Linear regression

$$\hat{\beta} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2}, \quad \hat{\alpha} = \bar{Y} - \hat{\beta}\bar{X}. \quad (1)$$

- We are estimating these quantities, hence $\hat{\beta}$ not β
- These estimates minimize the sum of squared errors

Linear regression: Theory

- Why do we care about the sum of squared errors when we fit a line?
- One reason is statistical. Imagine there was a “true” value of α and β and that the Y_i values were generated by the following process:
- Set $Z_i = \beta X_i + \alpha$
- Set $Y_i = Z_i + \varepsilon_i$, where $\varepsilon_1, \dots, \varepsilon_n$ are independent random variables following a normal distribution with mean 0 and standard deviation σ

-
- Under this generation process, the OLS solution $\hat{\alpha}, \hat{\beta}$ is the *maximum likelihood estimate* for α and β
 - This is another way of saying these values are the best in some sense
 - This is a different sense to minimizing the SSE in general, happen to give the same result here
 - In nature, many variables are independent and normally distributed (this is because of the *central limit theorem*)

Linear regression: Theory

- Assume we have input $X = (X_1, \dots, X_n)$ and output $Y = (Y_1, \dots, Y_n)$ (example: X_i is month index, Y_i is demand for product A)
- We assume that the “true” demand is $f(X_i) = \beta X_i + \alpha$
- The “observed” demand is a noisy version of the “true” demand: $\alpha + \beta X_i + \epsilon_i$
- Here $\epsilon_i \sim N(0, \sigma) \Rightarrow Y_i - f(X_i) \sim N(0, \sigma)$
- In regression, the quantities $\varepsilon = Y_i - f(X_i)$ are called residuals

Linear regression: Theory

- The log likelihood of parameters α, β given X and Y is:

$$\log \prod_{i=1}^n \Pr(Y_i | X_i, \alpha, \beta) \tag{2}$$

$$= \sum_{i=1}^n \log \frac{1}{\sigma \sqrt{2\pi}} \exp \left(-\frac{1}{2} \left(\frac{f(X_i) - Y_i}{\sigma} \right)^2 \right) \tag{3}$$

$$\propto - \sum_{i=1}^n (f(X_i) - Y_i)^2 + K \tag{4}$$

Some more details

- The log likelihood is

$$\sum_{i=1}^n \log \frac{1}{\sigma\sqrt{2\pi}} \exp \left(-\frac{1}{2} \left(\frac{f(X_i) - Y_i}{\sigma} \right)^2 \right)$$

- This is because $Y_i - f(X_i) \sim N(0, \sigma)$ under this model
 - The log likelihood is the log of the pdf of a Normal distribution
-

- The log likelihood is maximized when the sum of squared errors is minimized
- Therefore, finding regression parameters that minimize the sum of squared errors is the same as assuming the residuals are normally distributed (with equal standard deviation) and finding the maximum likelihood solution for the parameters

Linear regression: Theory

- We only care about the parts which depend on α and β , hence why the log likelihood is

$$-\sum_{i=1}^n (f(X_i) - Y_i)^2 + K = -\sum_{i=1}^n (\alpha + \beta X_i - Y_i)^2 + K$$

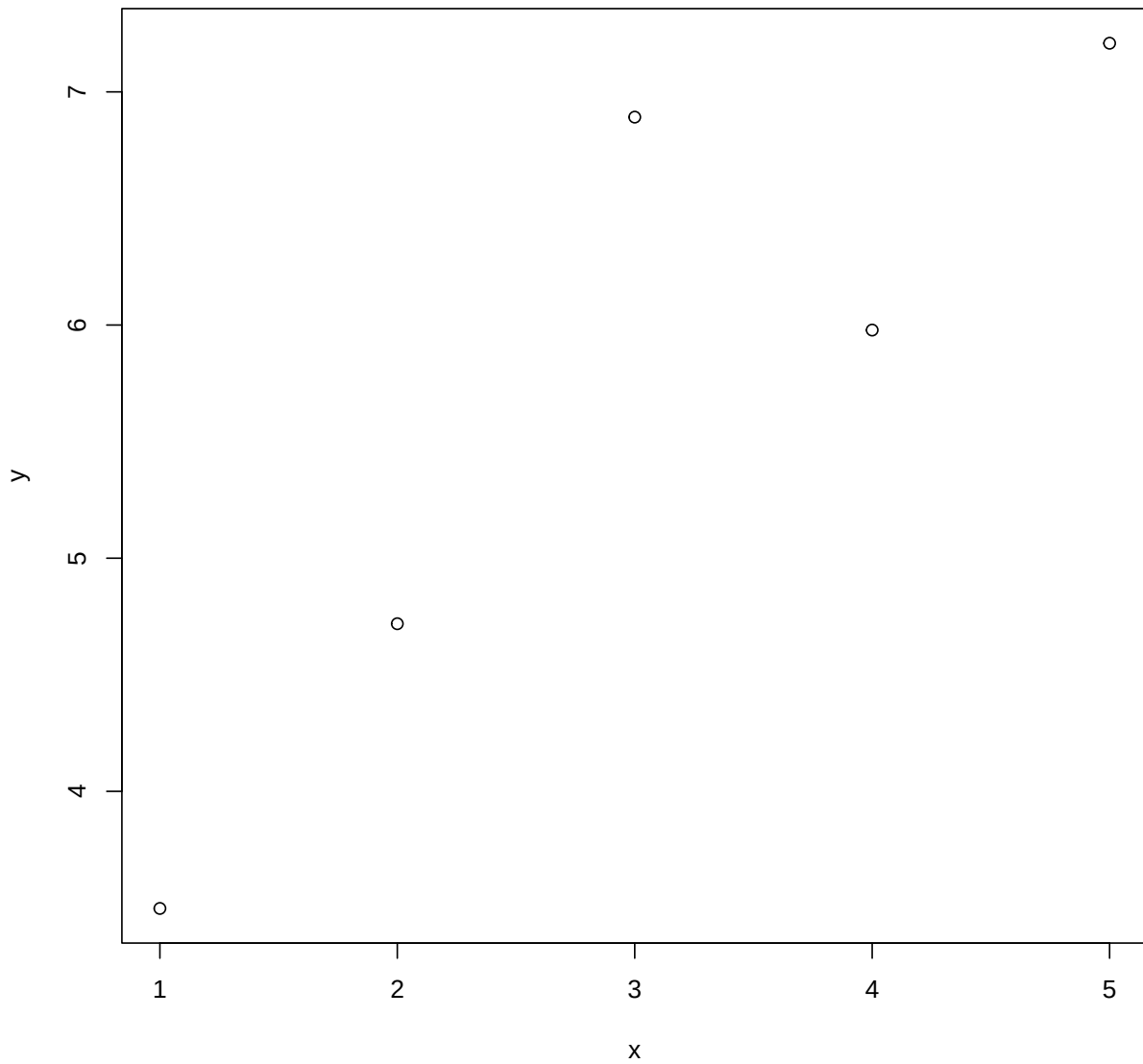
- Maximising the negative of the SSE
 - The same as minimizing the SSE!

Linear regression: Example

- Consider the following simulated dataset:

```
set.seed(20260211)
x <- 1:5
alpha <- 2
beta <- 1
epsilon <- rnorm(5) ## what is sigma here?
y <- beta * x + alpha + epsilon
```

```
plot(x, y, xlab = "x", ylab = "y")
```



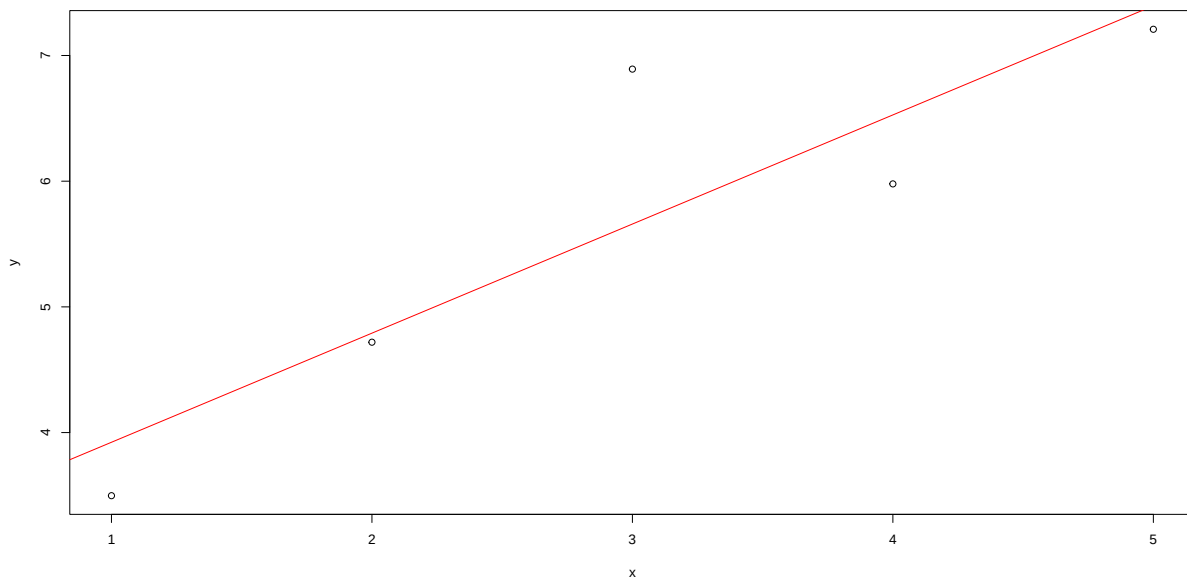
Linear regression: Example

- We have the following OLS solution:


```

model <- lm(y ~ x, data = data.frame(x = x, y = y))
alpha0 <- summary(model)$coefficients[1, 1]
beta0 <- summary(model)$coefficients[2, 1]
plot(x, y, xlab = "x", ylab = "y")
abline(alpha0, beta0, col = "red")

```



Linear regression: Example

- We compare the OLS solution with a “by hand” minimization of the sum of squared errors:

```

my_lm <- function(x, y) {
  xm <- mean(x)
  ym <- mean(y)
  beta <- sum((x - xm) * (y - ym)) / sum((x - xm)^2)
  alpha <- ym - beta * xm
  return(list(alpha = alpha, beta = beta))
}
result <- my_lm(x, y)
alpha1 <- result$alpha
beta1 <- result$beta
print(c(alpha0, beta0))

```

```
[1] 3.0546440 0.8681461
```

```
print(c(alpha1, beta1))
```

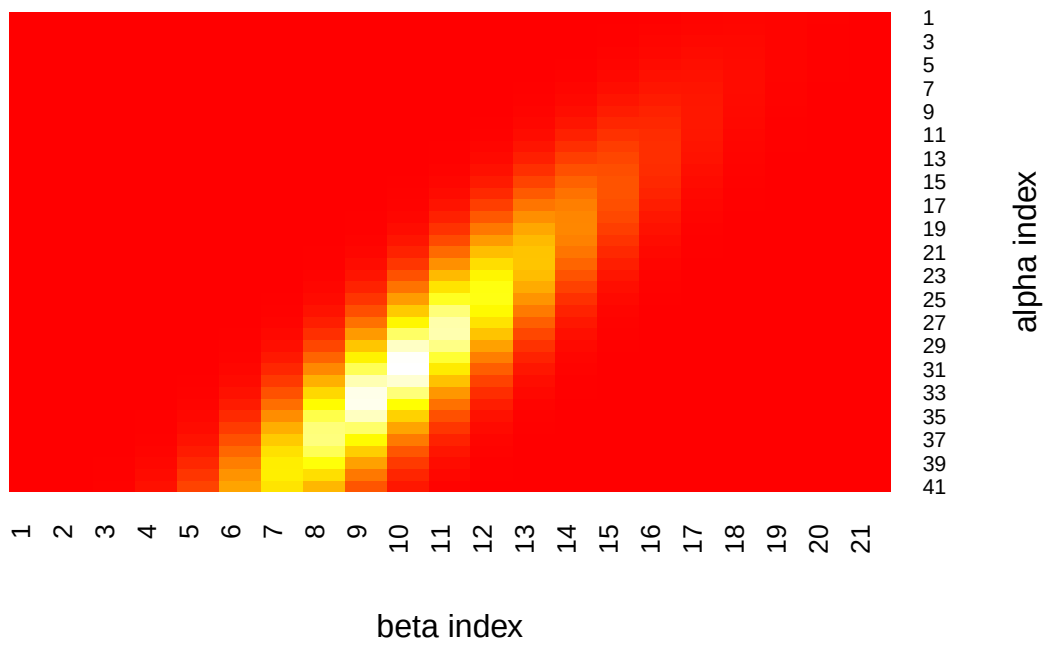
```
[1] 3.0546440 0.8681461
```

Linear regression: Example

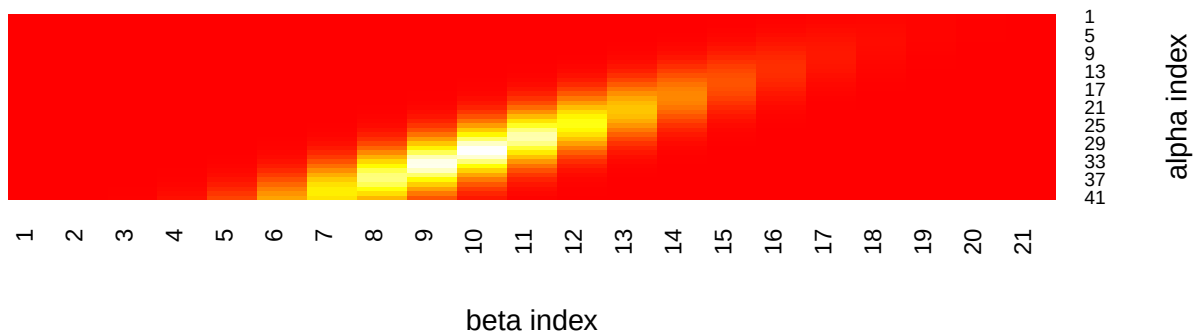
- We calculate the (log) likelihood over a range of parameters under a model with standard normal residuals:

```
as <- seq(0, 4, by = 0.1)
bs <- seq(0, 2, by = 0.1)
result <- matrix(NA, length(as), length(bs))
lik <- function(a, b) {
  prod((dnorm(b * x + a - y)))
}
for (i in 1:length(as)) {
  for (j in 1:length(bs)) {
    result[i, j] <- lik(as[i], bs[j])
  }
}
```

```
gplots::heatmap.2(result,
  trace = "none", dendrogram = "none",
  Rowv = FALSE, Colv = FALSE, key = FALSE,
  symbreaks = FALSE, col = heat.colors(n = 1024),
  lhei = c(0.1, 2), lwid = c(0.1, 7),
  ylab = "alpha index", xlab = "beta index"
)
```



Linear regression: Example



```
max(result)
```

```
[1] 0.003607823
```

```
lik(alpha1, beta1)
```

```
[1] 0.003641383
```

- The OLS estimate from lm is the highest value on the heatmap

Problem

- Repeat this exercise, but instead of plotting the likelihood here, plot the Sum of Squared Error (using the function we saw last week)
- Does the plot look any different?

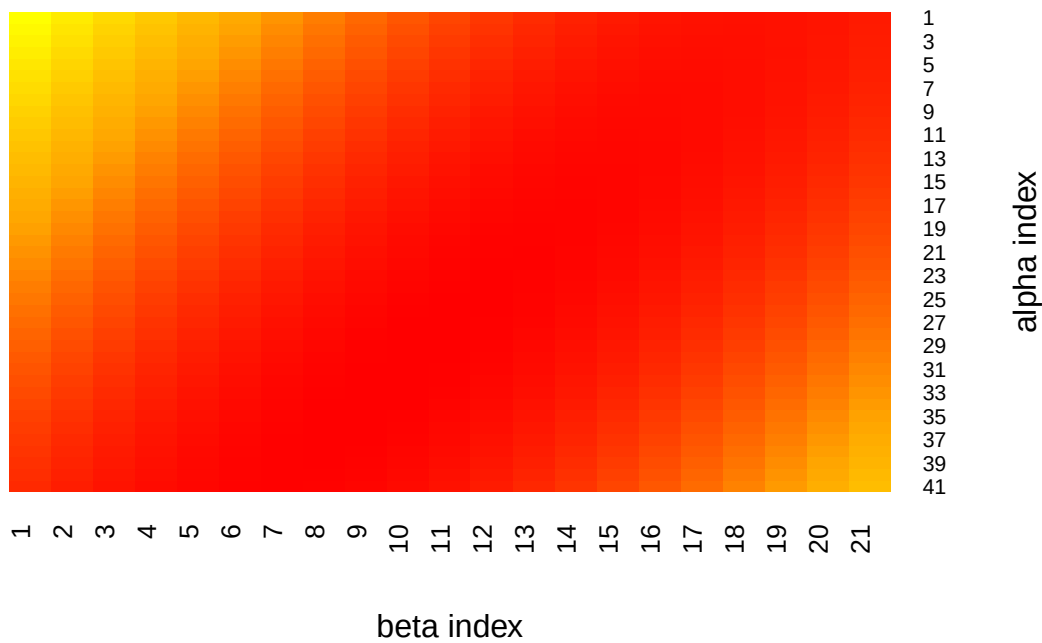
Solution

```
as <- seq(0, 4, by = 0.1)
bs <- seq(0, 2, by = 0.1)
result_sse <- matrix(NA, length(as), length(bs))
sse <- function(a, b) {
  sum( (y - a - b * x)^2)
}
for (i in 1:length(as)) {
  for (j in 1:length(bs)) {
    result_sse[i, j] <- sse(as[i], bs[j])
  }
}
```

Solution

I changed the color scheme, it (maybe?) looks slightly better

```
heatmap.2(result_sse,
  trace = "none", dendrogram = "none",
  Rowv = FALSE, Colv = FALSE, key = FALSE,
  symbreaks = FALSE,
  col = colorRampPalette(c("red", "orange", "yellow"))(1024),
  lhei = c(0.1, 2), lwid = c(0.1, 7),
  ylab = "alpha index", xlab = "beta index"
)
```



More Data

- Here $\alpha = 2$ and $\beta = 1$ while $\hat{\alpha} \approx 3$ and $\hat{\beta} \approx 0.86$
- If we had a larger sample our estimates will get closer to the true values (on average)

Larger Simulation

```
set.seed(20260211)
x <- 1:500
alpha <- 2
beta <- 1
epsilon <- rnorm(5) ## what is sigma here?
y <- beta * x + alpha + epsilon
result <- my_lm(x, y)
alpha1_large <- result$alpha
beta1_large <- result$beta
print(c(alpha1_large, beta1_large))
```

```
[1] 2.6622531 0.9999873
```

More regression

- If you were actually using regression at Z Co. to determine how much stock to buy, you would need to check if your model is any good!
- Prediction (and model) evaluation is a key part of the data science workflow
- We will come back to this (in quite a bit of detail) in the second half of this course!

Acquiring Data using APIs

Data acquisition

- To develop products, portfolios and projects using data science, we need data!
- Download data from public repositories
- Use private data (i.e., from your company, or from a scientific collaboration)
- Use APIs (interact with an App or service)

Data acquisition

- [UCI Machine Learning Repository](#)

archive.ics.uci.edu/ml/index.php

UCI Machine Learning Repository
Center for Machine Learning and Intelligent Systems

About Citation Policy Donate a Data Set Contact

Search

Repository Web

View ALL Data Sets

Check out the beta version of the new UCI Machine Learning Repository we are currently testing! [Contact us](#) if you have any issues, questions, or concerns. [Click here to try out the new site.](#)

Welcome to the UC Irvine Machine Learning Repository!

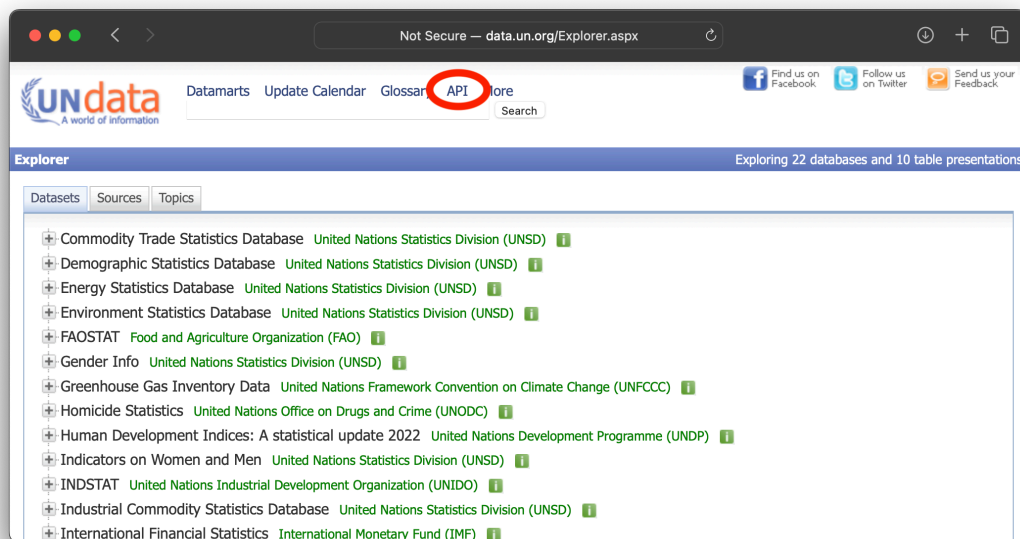
We currently maintain 622 data sets as a service to the machine learning community. You may [view all data sets](#) through our searchable interface. For a general overview of the Repository, please visit our [About page](#). For information about citing data sets in publications, please read our [citation policy](#). If you wish to donate a data set, please consult our [donation policy](#). For any other questions, feel free to [contact the Repository librarians](#).

Supported By: In Collaboration With:

Latest News:	Newest Data Sets:	Most Popular Data Sets (hits since 2007):
09-24-2018: Welcome to the new Repository admins Dheeru Dua and Efi Karra Taniskidou! 04-04-2013: Welcome to the new Repository admins Kevin Bache and Moshe Lichman! 03-01-2010: Note from donor regarding Netflix data 10-16-2009: Two new data sets have been added. 09-14-2009: Several data sets have been added. 03-24-2008: New data sets have been added! 06-25-2007: Two new data sets have been added: UJI	06-05-2021: Average Localization Error (ALE) in sensor node localization process in WSNs 05-25-2021: 9mers from culipdb 05-18-2021: TamilSentiMix	5158346: Iris 2693176: Adult 2201526: Dry Bean Dataset

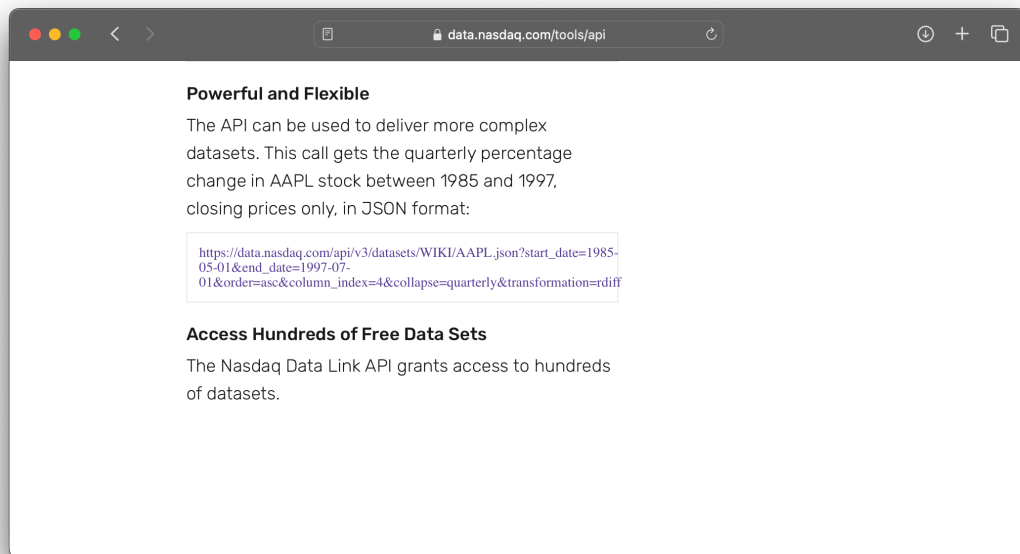
Data aquisition

- UNdata: A world of information



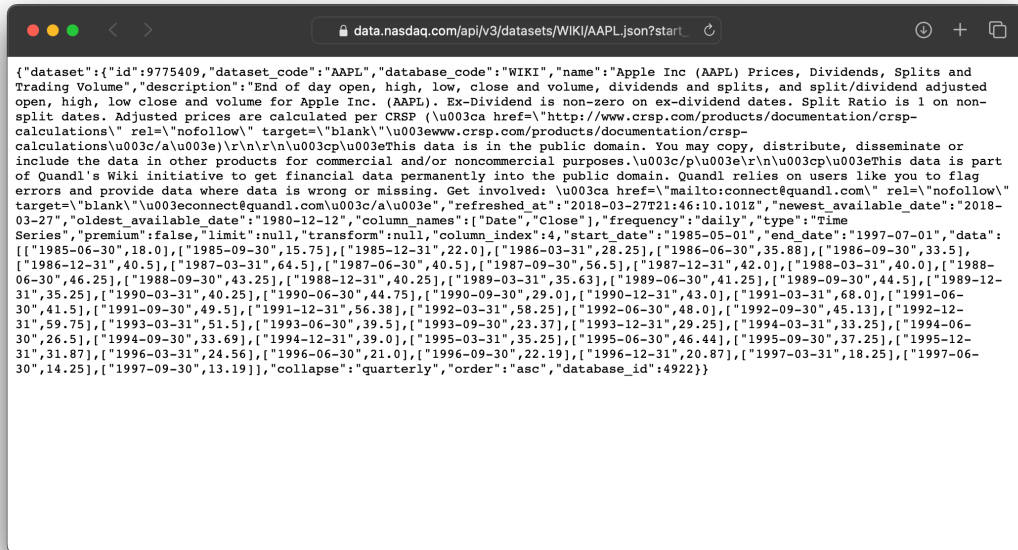
Data acquisition

- NASDAQ API



Data aquisition

- NASDAQ API



The screenshot shows a web browser window with the address bar displaying `data.nasdaq.com/api/v3/datasets/WIKI/AAPL.json?start=`. The main content area displays a JSON response from the API. The JSON object contains metadata about the dataset, including its ID, code, name, and description. It also includes a list of column names and a large array of data points, each representing a trading day for Apple Inc. (AAPL) from 1985 to 1997. The data points are structured as arrays containing date, open, high, low, close, volume, and split/dividend information.

API: Application programming interface

- Interact with an App, service or data source
- Agnostic to programming language used (the language “interfaces” with the service by calling commands)
- Usually done over the internet

An application programming interface (API) is a way for two or more computer programs to communicate with each other

— Wikipedia

Example: NASDAQ

- For example, in the NASDAQ API, a command is run by going to a URL (universal resource locator). The content of the “website” at the URL *is* the requested data

```
https://data.nasdaq.com/api/v3/datasets/WIKI/AAPL.json?start_date=1985-05-01&end_date=1997-07-01&order=asc&column_index=4&collapse=quarterly&transformation=rdiff
```

- (What does rdiff mean here?)

API documentation

- Often, APIs exist so that the service is used (either for free, or through subscription). So, they are often well documented

The screenshot shows a web browser window displaying the Nasdaq Data Link API documentation. The left sidebar contains a navigation menu with sections: DOCUMENTATION, API USAGE, TIME-SERIES, TABLES, and ANALYSIS TOOLS. The 'PARAMETERS' section is highlighted in blue. The main content area features a table with parameters: transform, no, string, none, diff, rdiff, rdiff_from, cumul, and normalize. Below the table, there is a 'Collapse' section explaining that the last observation is returned for any given time period. A 'Transform' section states that available transformations are listed in the table below. The 'Transformations' table has three columns: Name, Effect, and Formula.

Name	Effect	Formula
none	no effect	$z[t] = y[t]$
diff	row-on-row change	$z[t] = y[t] - y[t-1]$
rdiff	row-on-row % change	$z[t] = (y[t] - y[t-1]) / y[t-1]$

Retrieving URLs in R

- Let's use the (free) Open Meteo API to get the max and min forecast temperature at SFU for the next 7 days

```
library(httr)
# Open-Meteo API (No API key required)
url <- "https://api.open-meteo.com/v1/forecast?latitude=49.2800&longitude=-122.9150&daily=temperatures"

response <- GET(url)
print(response)
```

```
Response [https://api.open-meteo.com/v1/forecast?latitude=49.2800&longitude=-122.9150&daily=temperatures]
  Date: 2026-02-11 00:31
  Status: 200
  Content-Type: application/json; charset=utf-8
  Size: 496 B
```

Retrieving URLs in R

- We convert to JSON, and explore to find out how to index the data in the list

```
json_text <- content(response, as = "text", encoding = "UTF-8")
json_text
```

```
[1] "{\"latitude\":49.279037,\"longitude\":-122.90999,\"generationtime_ms\":0.05984306335449\"}
```

```
library(rjson)
parsed <- fromJSON(json_text)
parsed
```

```
$latitude
[1] 49.27904
```

```
$longitude
[1] -122.91
```

```
$generationtime_ms
[1] 0.05984306
```

```
$utc_offset_seconds
[1] -28800
```

```

$timezone
[1] "America/Vancouver"

$timezone_abbreviation
[1] "GMT-8"

$elevation
[1] 349

$daily_units
$daily_units$time
[1] "iso8601"

$daily_units$temperature_2m_max
[1] "°C"

$daily_units$temperature_2m_min
[1] "°C"

$daily
$daily$time
[1] "2026-02-10" "2026-02-11" "2026-02-12" "2026-02-13" "2026-02-14"
[6] "2026-02-15" "2026-02-16"

$daily$temperature_2m_max
[1] 9.3 8.9 7.6 4.3 4.9 5.3 6.6

$daily$temperature_2m_min
[1] 3.4 2.3 1.8 1.3 0.0 -2.0 -2.1

```

Retrieving from a URL in R

- Once we understand how to index into the list, we can convert to a dataframe:

```

weather <- data.frame(
  Date = parsed$daily$time,
  Max_Temp = parsed$daily$temperature_2m_max,
  Min_Temp = parsed$daily$temperature_2m_min
)
weather

```

	Date	Max_Temp	Min_Temp
1	2026-02-10	9.3	3.4
2	2026-02-11	8.9	2.3
3	2026-02-12	7.6	1.8
4	2026-02-13	4.3	1.3
5	2026-02-14	4.9	0.0
6	2026-02-15	5.3	-2.0
7	2026-02-16	6.6	-2.1

R packages for APIs

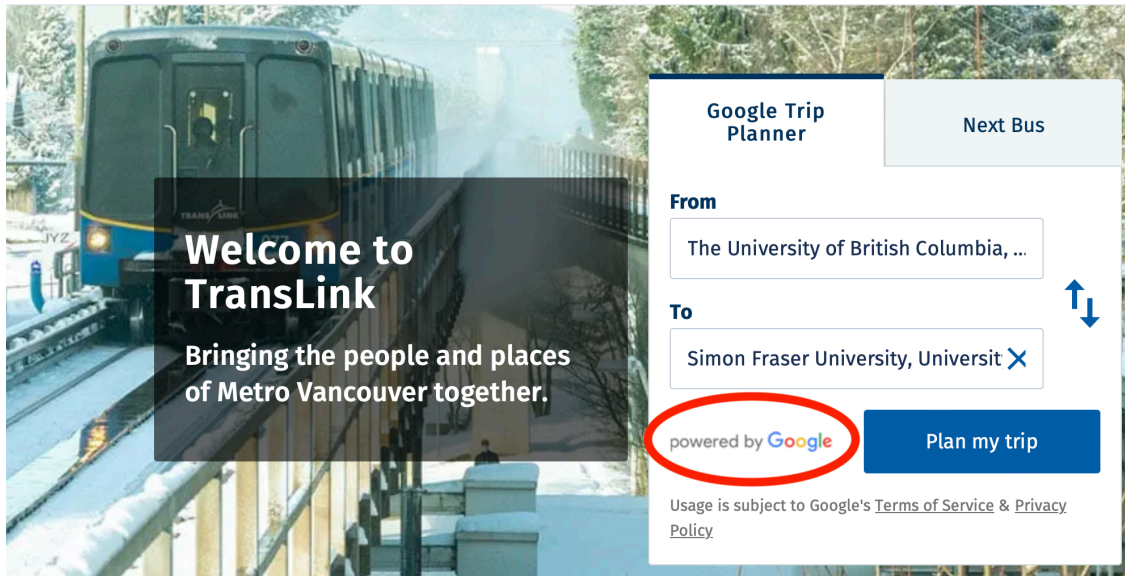
- While APIs are programming language agnostic, many APIs have libraries in a variety of languages (these may construct URLs and parse results).
- Open Meteo has an R package `openmeteo`

```
library(openmeteo)
weather_forecast(location = c(49.2800, -122.9150),
  daily = c("temperature_2m_max", "temperature_2m_min"))
```

```
weather_forecast(location = c(49.2800, -122.9150),
  daily = c("temperature_2m_max", "temperature_2m_min"))
```

```
# A tibble: 7 x 3
  date      daily_temperature_2m_max daily_temperature_2m_min
  <date>          <dbl>          <dbl>
1 2026-02-10      9.3            3.4
2 2026-02-11      8.9            2.3
3 2026-02-12      7.6            1.8
4 2026-02-13      4.3            1.3
5 2026-02-14      4.9             0
6 2026-02-15      5.3           -2
7 2026-02-16      6.6           -2.1
```

Example: Google maps



The image shows a screenshot of the TransLink website with a Google Trip Planner overlay. The background is a photograph of a blue and white TransLink train on a snowy track. A dark grey semi-transparent box is overlaid on the left side of the image, containing the TransLink logo and the text "Welcome to TransLink" and "Bringing the people and places of Metro Vancouver together." On the right side, the Google Trip Planner interface is visible. It has a white background and a blue header with the text "Google Trip Planner" and a "Next Bus" button. Below the header, there are two input fields: "From" with the text "The University of British Columbia, ..." and "To" with the text "Simon Fraser University, Universit X". A blue double-headed arrow is positioned between the two input fields. Below the input fields, there is a red circle around the text "powered by Google". To the right of this circle is a blue button with the text "Plan my trip". At the bottom of the interface, there is a small text line: "Usage is subject to Google's [Terms of Service](#) & [Privacy Policy](#)".

Welcome to TransLink
Bringing the people and places of Metro Vancouver together.

Google Trip Planner **Next Bus**

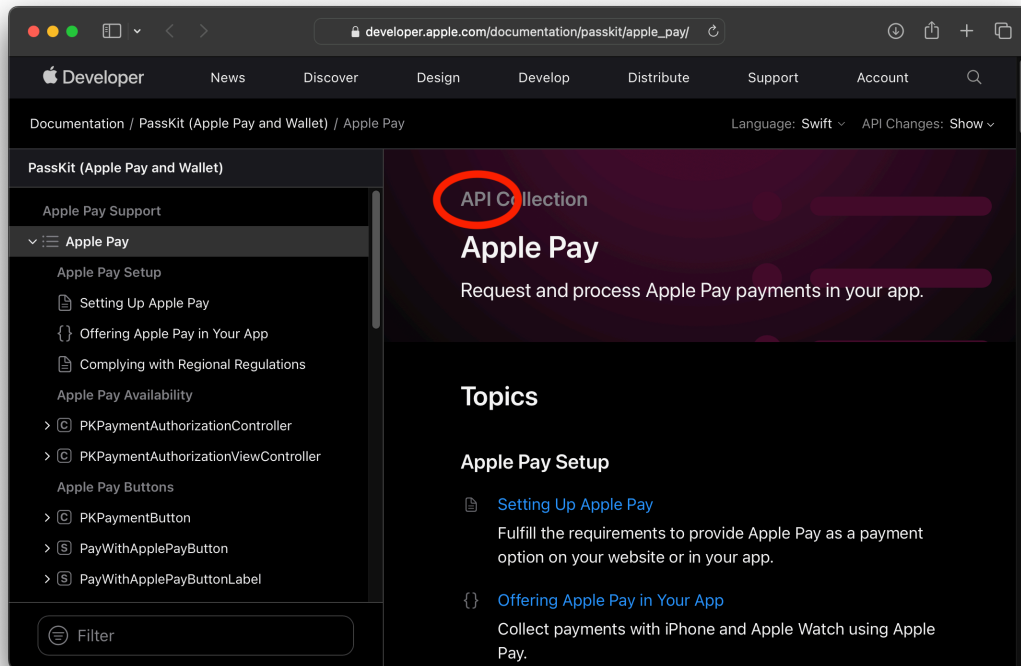
From
The University of British Columbia, ...

To
Simon Fraser University, Universit X

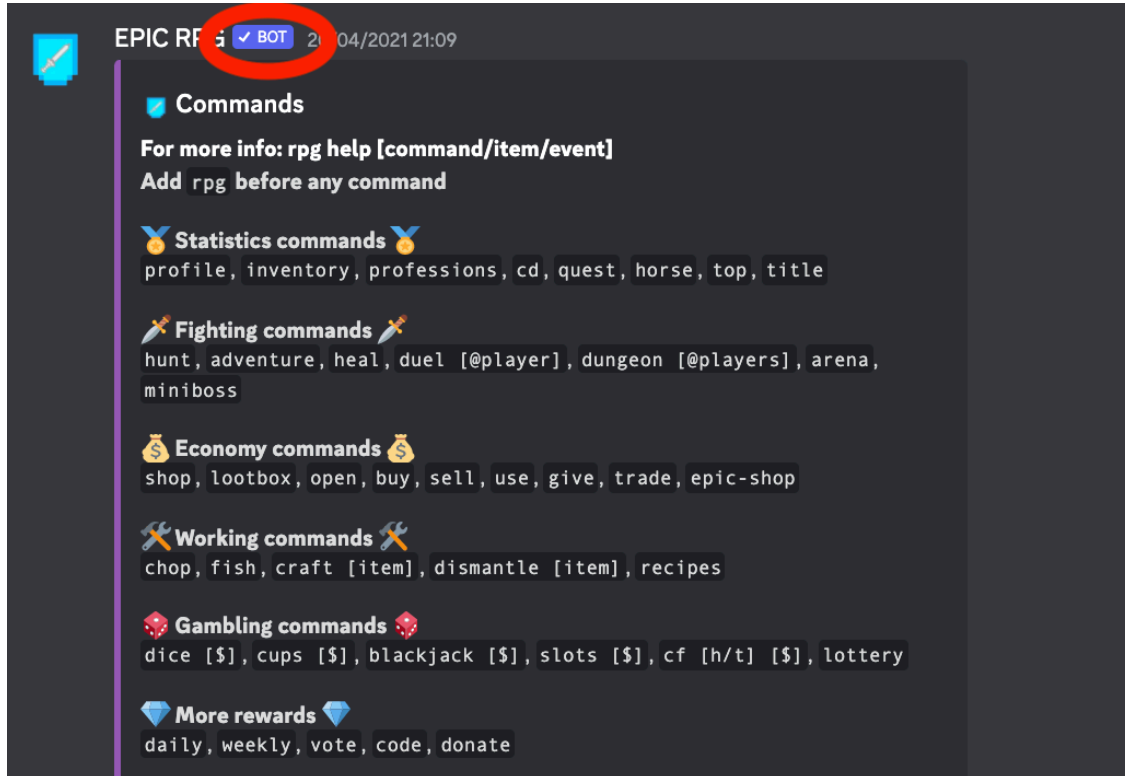
powered by Google **Plan my trip**

Usage is subject to Google's [Terms of Service](#) & [Privacy Policy](#)

Example: Apple pay



Example: Discord



Example: eBird

- eBird is a database of bird sightings maintained by the Cornell Lab of Ornithology at Cornell University, Ithica
- more than 100m complete “checklists” by citizen scientists (recreational and professional), >1b bird observations
- eBird has an [API](#), and the API has an R package: [rebird](#)

eBird

Using the API

- To use this (all?) APIs you need to have an API Key
- This is unique to you, so that the API owner can track who is using the service
 - It costs money to host this data and provide access to it!
- To get an API for Google Maps, you need to give Google your credit card details

eBird API Key

- To get an API Key for eBird you need to create an account
- Means you can report bird sightings also!
- [Good for your mental health!](#)



Figure 1: A Ring Billed Gull

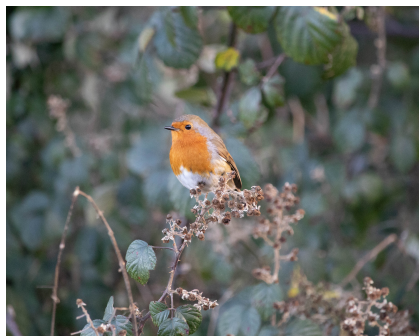


Figure 2: A Eurasian Robin

First ever sighting in Canada

Using the API Key

- When you connect to the API (through R) you need to pass it your API key
- Two ways to do this
 - Create an environment variable (better, but harder)
 - Pass it in directly in each call (easier, initially)

eBird API

- Working from the eBird manual, we will develop code to explore bird counts in the provinces and territories of Canada.
- Regions in the eBird database are sorted by “subnational1” and “subnational2” regions.
- We want to find provinces, so we find the function `ebirdsubregionlist` in the manual that will give us “subnational1” codes in Canada:

-
- To run this code you will need your own API key!
 - I have stored mine as an environment variable, but you would edit your code here to yours

```
library(rebird)
subregions <- ebirdsubregionlist(
  regionType = "subnational1",
  parentRegionCode = "CA", key = API_KEY
)

print(subregions)
```

```
# A tibble: 13 x 2
  code name
  <chr> <chr>
1 CA-AB Alberta
2 CA-BC British Columbia
3 CA-MB Manitoba
4 CA-NB New Brunswick
5 CA-NL Newfoundland and Labrador
6 CA-NT Northwest Territories
7 CA-NS Nova Scotia
8 CA-NU Nunavut
9 CA-ON Ontario
10 CA-PE Prince Edward Island
11 CA-QC Quebec
12 CA-SK Saskatchewan
13 CA-YT Yukon Territory
```

“Tibble” to vector

- Converting “A tibble: 13 × 2” to a vector:

```
as.vector(subregions[,1])
```

```
$code
[1] "CA-AB" "CA-BC" "CA-MB" "CA-NB" "CA-NL" "CA-NT" "CA-NS" "CA-NU" "CA-ON"
[10] "CA-PE" "CA-QC" "CA-SK" "CA-YT"
```

```
class(as.vector(subregions[,1]))
```

```
[1] "list"
```

```
as.vector(subregions[,1])$code
```

```
[1] "CA-AB" "CA-BC" "CA-MB" "CA-NB" "CA-NL" "CA-NT" "CA-NS" "CA-NU" "CA-ON"  
[10] "CA-PE" "CA-QC" "CA-SK" "CA-YT"
```

eBird API: Species list

- Again, from eBird API:

```
species <- ebirdregionspecies("CA", key = API_KEY)  
sp <- as.vector(species)$speciesCode  
print(sp[1:10])
```

```
[1] "emu1"      "bbwduc"    "fuwduc"    "bahgoo"    "empgoo"    "snogoo"    "rosgoo"  
[8] "sxrgoo1"   "gragoo"    "swagoo1"
```

- These names not very clear

Problem

- We want to get count data for 4 species, which have the common names: Canada Goose, Barred Owl, Bald Eagle, House Sparrow
- To get information for these species, we need to know the scientific name, which you need to figure out from `rebird::ebirdtaxonomy()`
- You don't need an API key to do this!

Solution

```
comm_names <- c("Canada Goose", "Barred Owl", "House Sparrow",  
               "Bald Eagle")  
  
bird_info <- rebird::ebirdtaxonomy()  
  
info <- bird_info[bird_info$comName %in% comm_names, ]  
info$speciesCode
```

```
[1] "cangoo" "baleag" "brdowl" "houspa"
```

eBird API: By region

- Want to loop over these species and the regions of Canada, count how many observations of each species in the last 14 days

```
num_regions <- nrow(subregions)
curr_species <- info$speciesCode
species_counts <- matrix(NA,
                          nrow = length(curr_species),
                          ncol = num_regions)
for(i in 1:num_regions){
  for(j in 1:length(curr_species)){
    query <- ebirdregion(loc = subregions[i, 1],
                        species = curr_species[j],
                        key = API_KEY)
    species_counts[j, i] <- sum(query$howMany, na.rm = TRUE)
  }
}
species_counts <- as.data.frame(species_counts)
colnames(species_counts) <- subregions$code
final_result <- cbind(code = curr_species, species_counts)
head(final_result)
```

	code	CA-AB	CA-BC	CA-MB	CA-NB	CA-NL	CA-NT	CA-NS	CA-NU	CA-ON	CA-PE	CA-QC
1	cangoo	19498	22611	0	24	415	0	19287	0	43308	367	172
2	baleag	260	3948	66	144	52	0	609	0	1196	26	429
3	brdowl	4	91	5	6	0	0	20	0	142	0	104
4	houspa	3693	2353	979	117	150	28	125	0	6644	24	3648
	CA-SK	CA-YT										
1	354	0										
2	10	6										
3	0	0										
4	1321	20										

```
species_counts <- as.data.frame(species_counts)
colnames(species_counts) <- subregions$code

final_result <- cbind(code = curr_species, species_counts)
head(final_result)
```

	code	CA-AB	CA-BC	CA-MB	CA-NB	CA-NL	CA-NT	CA-NS	CA-NU	CA-ON	CA-PE	CA-QC
1	cangoo	19498	22611	0	24	415	0	19287	0	43308	367	172
2	baleag	260	3948	66	144	52	0	609	0	1196	26	429
3	brdowl	4	91	5	6	0	0	20	0	142	0	104
4	houspa	3693	2353	979	117	150	28	125	0	6644	24	3648
		CA-SK	CA-YT									
1		354	0									
2		10	6									
3		0	0									
4		1321	20									

eBird API: Preprocessed dataset

- CSV of similar output on Canvas for you to use

```
ebird30 <- read.csv("ebird30.csv")
print(dim(ebird30))
```

```
[1] 883 14
```

```
print(colnames(ebird30)[2:4])
```

```
[1] "CA.AB" "CA.BC" "CA.MB"
```

```
names <- read.csv("names.csv")
print(dim(names))
```

```
[1] 883 3
```

```
print(colnames(names))
```

```
[1] "Code" "Common.Name" "Scientific.Name"
```

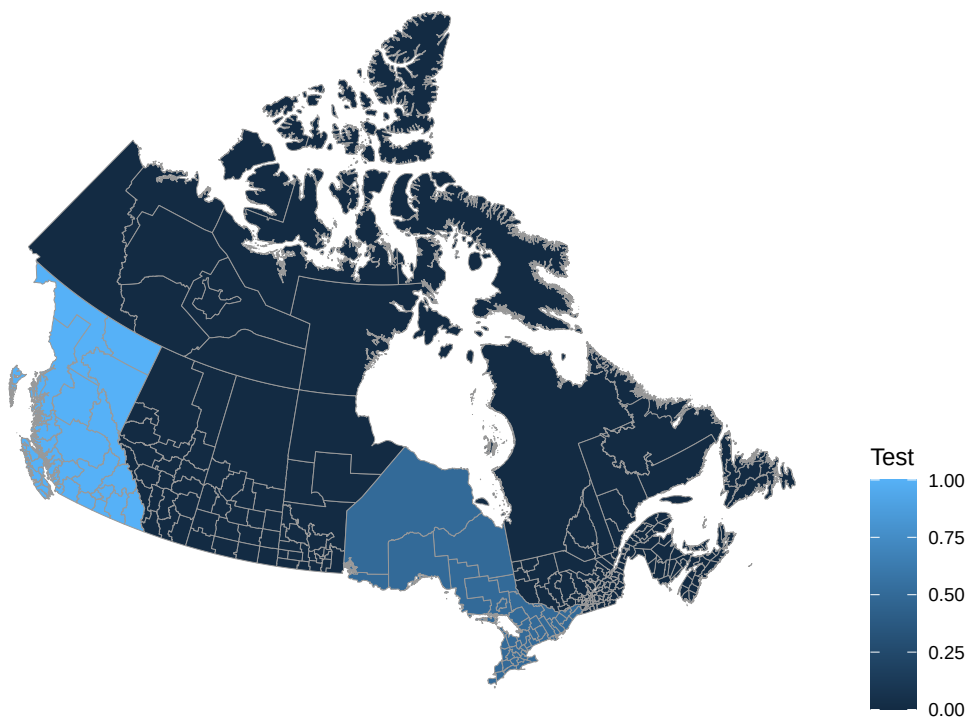
Drawing a map of Canada

- I've provided a script `canada.R` with a function `canada` that makes a heatmap for Canada:
- We can load the function from that in using `source`

```
source('canada.R')
heat <- list("CA.MB" = 0, "CA.BC" = 1, "CA.AB" = 0,
            "CA.SK" = 0, "CA.ON" = 0.5, "CA.QC" = 0,
            "CA.NL" = 0, "CA.NS" = 0, "CA.NB" = 0,
            "CA.PE" = 0, "CA.NT" = 0, "CA.NU" = 0,
            "CA.YT" = 0)
title <- "Test"
```

Drawing a map of Canada

```
canada(title, heat)
```



Heatmaps for birds

This can be combined with the eBird data to provide data visualizations:

```

heat <- list()
rownames(ebird30) <- ebird30$Code
for (i in 1:13) {
  heat[[colnames(ebird30)[i+1]]] = ebird30["houspa", colnames(ebird30)[i+1]]
}

heat[1:3]

```

```

$CA.AB
[1] 4223

```

```

$CA.BC
[1] 2981

```

```

$CA.MB
[1] 1007

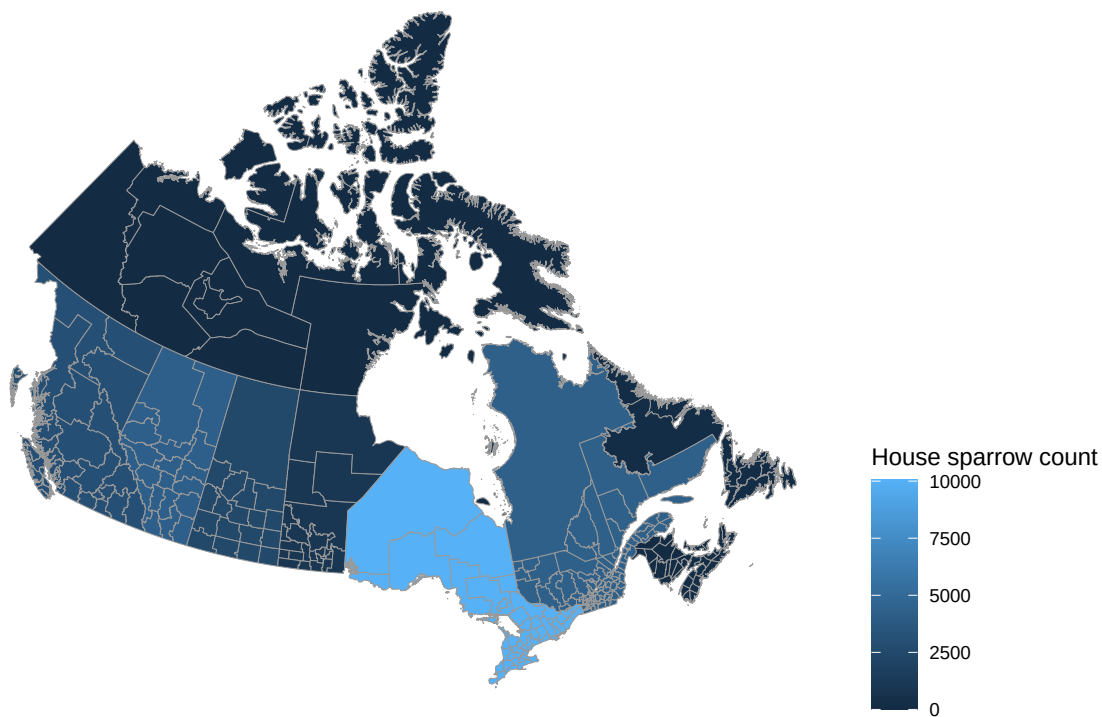
```

Heatmaps for birds

```

canada("House sparrow count", heat)

```



Quick Midterm Review

Instructions

- Review the sample midterm carefully, particularly instructions
- You can only access RStudio except to view class slides or create/submit your pdf

Basic R Coding and Functions

```
df <- data.frame(V1 = c(1, 2), V2 = c(3, 4), V3 = c(5, 6))
df$V1
```

```
[1] 1 2
```

```
df[, c("V1", "V3")]
```

```
  V1 V3
1  1  5
2  2  6
```

```
df[, c(1, 1, 2)]
```

```
  V1 V1.1 V2
1  1     1  3
2  2     2  4
```

```
v1 <- c(0, 1, 1)
class(v1)
```

```
[1] "numeric"
```

```
v2 <- c(FALSE, TRUE, TRUE)
class(v2)
```

```
[1] "logical"
```

```
df[, v1]
```

```
  V1 V1.1  
1  1    1  
2  2    2
```

```
df[, v2]
```

```
  V2 V3  
1  3  5  
2  4  6
```

Function Syntax

```
simp_function <- function(x) {  
  y <- sum(x ^2)  
  y  
}  
  
simp_function(c(1, 2, 3))
```

```
[1] 14
```

Reading and Writing Data

- read.table and write.table and the options for these functions

```
write.table(df, file = "output.txt", sep = ",")
```

Linear Regression

- Find α and β to minimize the Sum of Squared Errors

```
x <- mtcars$mpg  
y <- mtcars$disp  
lm(y ~ x)
```

Call:

```
lm(formula = y ~ x)
```

Coefficients:

(Intercept)	x
580.88	-17.43

SQL: Structured Query Language

- Advantages of relational databases: Availability, Integrity, Security, Efficiency
- SQL is a language. SQLite is an implementation of the SQL language.
- To write to a database we use INSERT or UPDATE:

```
INSERT INTO Platonic VALUES ('octahedron', 8)
```

```
UPDATE Platonic SET Name = 'Octahedron' WHERE Name = 'octahedron'
```

```
CREATE TABLE vocabulary(word TEXT PRIMARY KEY, count INT DEFAULT 1);  
INSERT INTO vocabulary(word) VALUES('doorknob')  
ON CONFLICT(word) DO UPDATE SET count=count+1;
```

- To read from a database we use SELECT and WHERE:

```
SELECT * FROM Platonic WHERE Sides > 5
```

- Other options to select specific data based on matching certain properties

```
query <- sqlInterpolate(db, "SELECT SUM(Quantity) FROM sales  
                             WHERE StockCode = '71053' AND  
                             InvoiceDate LIKE ?month",  
  month = paste0(months[i], "%")  
)
```

- Also saw GROUP BY and AS

APIs

- APIs (Application Programming Interfaces) allow code to interact with services
- Example: The NASA API allows us to interact with NASA data.

```
library(httr)
library(imager)
url <- "https://api.nasa.gov/planetary/earth/imagery?lon=-123.116226&lat=49.246292&date=2022-01-01"
download.file(url, "image.png")
image <- load.image("image.png")
plot(image)
```

